

ЛЕКЦИЯ 6

Обеспечение безопасности ОС Unix (Linux)



Учебные вопросы:

1. Механизмы защиты ОС семейства Unix
2. Аудит в ОС семейства UNIX

Дополнительный материал:



01

Механизмы защиты ОС семейства Unix

Защита ОС семейства Unix в общем случае базируется на трех основных механизмах:

1. идентификации и аутентификация пользователя при входе в систему;
2. разграничении прав доступа к файловой системе, в основе которого лежит реализация дискреционной модели доступа;
3. аудит (регистрация событий).



Аутентификация в UNIX

- Список пользователей UNIX хранится в текстовом файле ***/etc/passwd***, формат которого допускает его редактирование обычным текстовым редактором.
- В большинстве современных UNIX-систем образы паролей хранятся в отдельном файле, также расположенном в каталоге ***/etc***
- чаще всего применяется имя ***/etc/shadow***.
-
- В некоторых UNIX-системах пароль каждого пользователя хранится в своем отдельном файле, например, в ALT Linux пароль пользователя **username** хранится в файле ***/etc/tcb/username/shadow***.

Каждая строка файла ***/etc/passwd*** соответствует одному пользователю или псевдопользователю.

Она включает в себя следующие семь полей, разделенных двоеточиями:

- – имя пользователя (login);
- – хэш-образ пароля пользователя или признак того, что хэш-образ пароля хранится в другом файле, общем для всех пользователей (*) или признак того, что хэш-образ пароля хранится в отдельном файле (x);
- – UID пользователя;
- – GID первичной группы пользователя;
- – комментарий к учетной записи пользователя (произвольная текстовая строка);
- – домашняя директория пользователя;
- – командный интерпретатор, который должен запускаться на терминалах, на которых работает данный пользователь (*для псевдопользователей обычно указывается /dev/null*).

Например, root:x:0:0:System Administrator:/root:/bin/bash.

- Управление списком пользователей, как правило, осуществляется не путем ручного редактирования соответствующих файлов, а с использованием специальных утилит командной строки **user add, userdel, usermod, groupadd, groupdel, groupmod**, либо графических утилит, оснасток панели управления и т. п.
- Хранение отпечатка пароля в файле */etc/passwd* применялось в старых версиях UNIX систем, при этом оставалась возможность оффлайн-подбора пароля. Учитывая, что содержимое файла */etc/passwd* имеет текстовый формат, а сам файл – общедоступен, такой способ хранения аутентификационной информации нельзя считать надежным.
- Поэтому для Linux были разработаны дополнительные механизмы защиты. 

Механизм теневых паролей

- **Файл */etc/passwd*** больше не содержит отпечатка пароля, вместо него помещается значение, указывающее, что отпечаток пароля хранится в специальном парольном файле. Права на чтение парольного файла ***/etc/shadow*** принадлежат только суперпользователю.
- Хранимые в файле отпечатки пользовательских паролей получены с помощью функции ***crypt***, допускающей как шифрование паролей с помощью любых методов, поддерживаемых криптографической библиотекой.
- Например, в Linux Debian/Ubuntu библиотека ***libcrypt*** поддерживает хэш, основанный на современном шифре Blowfish, или бесключевые хэш-функции с подмешиванием случайной информации – соли, при этом используется многократное (порядка 1000 раз) применение алгоритма, что замедляет вычисление отпечатка, а значит и снижает вероятность успешного подбора паролей
- Для реализации механизма теневых паролей в Linux используется набор программных средств ***Shadow Suite***. В большинстве современных дистрибутивов Linux механизм теневых паролей задействован по умолчанию.

Архитектура Pluggable Authentication Module (PAM)

- *Программные модули, входящие в подсистему аутентификации, делятся на две основные группы:*

1. клиенты (приложения и демоны), пользующиеся услугами PAM (login, passwd, rlogin, telnetd, ftpd и т.п.);
2. программные модули, предоставляющие услуги PAM. Обычно эти модули представляют собой библиотеки, размещаемые в директории /lib/security.

С точки зрения клиента аутентификации его взаимодействие с PAM реализуется посредством так называемых примитивов — системных вызовов, передающих управление соответствующим модулям PAM.

Поддерживаются шесть основных примитивов, сгруппированных в четыре подсистемы::

- `ram_authenticate` (подсистема `auth`) – аутентифицировать пользователя;
- `ramjsetcred` (подсистема `auth`) – авторизовать пользователя (установить UID, идентификаторы групп, квоты ресурсов и т.д.);
- `ram_acct_mgmt` (подсистема `account`) – проверить, доступна ли учетная запись пользователя для авторизации (не устарел ли пароль, не заблокирована ли учетная запись и т.п.);
- `ram_openjsession` (подсистема `session`) – начать сеанс работы пользователя с операционной системой;
- `ram_closejsession` (подсистема `session`) – завершить сеанс работы пользователя с операционной системой;
- `ram_chauthtok` (подсистема `password`) – назначить пользователю новые аутентификационные данные.



Архитектура Pluggable Authentication Module (PAM)

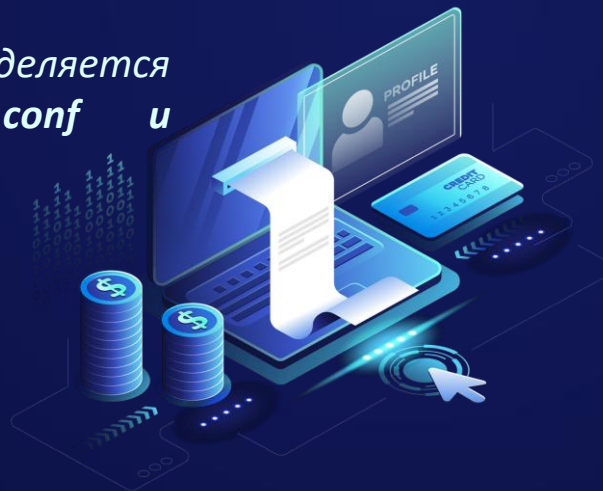
- *Программные модули, входящие в подсистему аутентификации, делятся на две основные группы:*

1. клиенты (приложения и демоны), пользующиеся услугами PAM (login, passwd, rlogin, telnetd, ftpd и т.п.);
2. программные модули, предоставляющие услуги PAM. Обычно эти модули представляют собой библиотеки, размещаемые в директории /lib/security.

С точки зрения клиента аутентификации его взаимодействие с PAM реализуется посредством так называемых примитивов — системных вызовов, передающих управление соответствующим модулям PAM.

Структура модулей PAM

1. Каждый модуль содержит функции-обработчики примитивов
2. Обработка вызовов клиента и принятие решений на основе результатов
3. Конфигурация определяется файлами `/etc/pam.conf` и `/etc/pam.d/`



Конфигурация PAM

1. В ранних версиях — один файл `/etc/pam.conf`
2. В новых версиях — индивидуальные файлы в `/etc/pam.d/` (например, `/etc/pam.d/sshd`)
3. Файл `/etc/pam.d/other` — конфигурация по умолчанию

Примеры настройки PAM

1. *Запрет выполнения команды su:*

```
auth requisite pam_deny.so
```

2. *Аутентификация через внешние носители (pam_usb.so) и биометрию (pam_lxprint.so)*
3. *Настройка схем аутентификации для пользователей и групп (pam_listfile.so)*

Гибкость и адаптация PAM

1. *Архитектура позволяет гибко настраивать аутентификацию*
2. *Установка необходимых модулей и их взаимодействие*
3. *Адаптация к различным технологиям и требованиям организации*

Сетевая защита

- Средства защиты:

- защищенная оболочка **ssh** для предотвращения атак, в которых для получения паролей используются анализаторы протоколов;
- программы **tcp_wrappers** для ограничения доступа к различным службам компьютера;
- межсетевой экран **netfilter** для ядер Linux, начиная с версии 2.4;
- сетевые сканеры для выявления уязвимых мест компьютера;
- демон **tcpd** для обнаружения попыток сканирования портов со стороны злоумышленников (в дополнение к этому средству полезно регулярно просматривать файлы системного журнала);
- система шифрования **PGP** (Pretty Good Privacy);
- программа **stelnnet** (защищенная версия хорошо известной программы telnet);
- программа **qmail** (защищенная доставка электронной почты);
- программа **ipfwadm** для настройки межсетевых экранов (firewall);
- режим проверки паролей входных соединений для систем, разрешающих подключение по внешним коммутируемым линиям связи или локальной сети.

Управление доступом к данным

- В UNIX объектами доступа являются **файлы, директории, ссылки, устройства и именованные каналы**. В современных версиях UNIX процессы также рассматриваются как объекты доступа.
- Атрибуты защиты объектов влияют на доступ отладчика и отправку сигналов.
- **Субъектами доступа в UNIX являются пользователи и группы пользователей**. Root имеет абсолютные полномочия и идентификатор 0. Пользователи и группы идентифицируются **UID и GID**.
- Доступ к объектам регулируется правами **Read, Write и Execute**.
- **Для каталогов:**
 - Read – просмотр содержимого,
 - Write – создание, переименование, удаление,
 - Execute – переход в каталог.
- **Для устройств:**
 - Read, Write – доступ к функциям драйвера,
 - Execute – не определено.

Управление доступом к данным

- В современных системах, включая Linux, привилегии называются возможностями (capabilities).
-
- **Примеры возможностей:**
- CAP.CHOWN – изменение владельца
- CAP.DAC.OVERRIDE – игнорирование ограничений доступа
- CAP.SETPCAP – управление привилегиями
- CAP_NET_BROADCAST – отправка широковещательных пакетов
- CAP_SYS_MODULE – загрузка/выгрузка модулей ядра
- CAP_SYS_RAWIO – низкоуровневый ввод-вывод
- CAP_SYS_BOOT – перезагрузка
- CAP_SYS_TIME – управление таймерами
- CAP_SYS_TTY_CONFIG – управление терминалами.

Управление доступом к данным

Каждому процессу UNIX присваиваются четыре числовых идентификатора:

- UID — идентификатор пользователя, породившего данный процесс;
- GID — идентификатор группы пользователя, породившего данный процесс;
- EUID — эффективный идентификатор пользователя;
- EGID — эффективный идентификатор группы пользователя.
- Обычно EUID совпадает с UID, а EGID — с GID.

Каждый объект UNIX может иметь атрибуты защиты, включающие в себя следующие три элемента:

- идентификатор (UID) владельца объекта;
- идентификатор группы (GID) владельца объекта;
- вектор доступа.
- Вектор доступа включает в себя следующие элементы:
- является ли объект каталогом — 1 бит (D);
- бит SUID;
- бит SGID;
- бит sticky;
- права доступа владельца — 3 бита (RWX);
- права доступа пользователей, входящих в группу владельца — 3 бита (RWX);
- — права доступа всех остальных пользователей — 3 бита (RWX).

Управление доступом к данным

Проверка прав доступа субъекта к объекту осуществляется по следующему алгоритму

1. Если EUID процесса равен нулю (т.е. процесс выполняется от имени суперпользователя), доступ к объекту предоставляется без каких бы то ни было дополнительных проверок.
2. Если EUID процесса совпадает с UID владельца объекта, права доступа, запрошенные процессом, сравниваются с правами, разрешенными владельцу данного объекта. Если все права, запрошенные процессом, разрешены владельцу объекта, доступ разрешен, в противном случае – запрещен.
3. Если EUID процесса отличается от UID владельца объекта, но EGID процесса совпадает с GID владельца объекта, права доступа, запрошенные процессом, сравниваются с правами, разрешенными пользователям, входящим в группу владельца объекта. Если все права, запрошенные процессом, разрешены группе владельца объекта, доступ разрешен, в противном случае – запрещен.
4. Если EUID процесса отличается от UID владельца объекта и EGID процесса отличается от GID владельца объекта, права доступа, запрошенные процессом, сравниваются с правами, разрешенными пользователям, не входящим в группу владельца объекта (всем остальным пользователям). Если все права, запрошенные процессом, разрешены пользователям, не входящим в группу владельца объекта, доступ разрешен, в противном случае – запрещен.

Управление доступом к данным

В современных UNIX-системах бит **sticky** применяется только к директориям, ограничивая удаление файлов и поддиректорий их владельцами. Суперпользователь **root** с неограниченными полномочиями создает риски для безопасности и надежности ОС. Ошибки при работе с командной строкой, например, команда **rm -rf /*.bak** с пробелом, могут уничтожить систему. Поэтому администраторы избегают использования **root** и применяют утилиту **su** для временного повышения прав.

Вместо **su** чаще используется **sudo**, позволяющая выполнять команды от имени суперпользователя или другого пользователя без ввода пароля (после однократного ввода). Конфигурация **sudo** описана в файле **/etc/sudoers**. Например, **root** может выполнять любые команды без пароля:

root ALL = NOPASSWD: ALL.

02

Аудит в ОС семейства UNIX

В современных UNIX-системах реализуются два подхода к организации аудита: традиционный с демонами `syslogd` и `klogd`, и с использованием интерфейсов подсистемы аудита Windows. Большинство систем поддерживают обе подсистемы, которые могут работать независимо или совместно.

Syslogd обрабатывает события от приложений и систем, сверяясь с политикой аудита, и регистрирует их **через `/dev/log`** (локально) или UDP-порт 514 (удаленно).

Klogd получает данные через **`/proc/kmsg`** или **`sys_syslog`**. Идентификаторы демонов хранятся в **`/var/run/syslogd.pid`** и **`/var/run/klogd.pid`**.

Политика аудита описывается в **`/etc/syslog.conf`**.



- В UNIX-системах большинство сообщений аудита записываются в файлы **/var/log/messages** или **/var/adm/log/messages**.
- Сообщения о попытках аутентификации обычно хранятся в файле **/var/log/auth.log** или **/var/adm/loginlog**.
- При изменении файла **syslog.conf** новая политика аудита применяется после перезагрузки демона **syslogd** или получения им сигнала **SIGHUP**. Этот сигнал интерпретируется как требование обновить политику аудита.
- **Журналы аудита** – текстовые файлы, которые можно просматривать стандартными утилитами или специализированными программами. Удаление устаревшей информации из журналов выполняется демоном **logrotate** по конфигурации в файле **/etc/logrotate.conf**.
- **Недостаток подсистемы аудита UNIX** – доступность демона **syslogd** для любых программ, что позволяет передавать сообщения, указывать источник и приоритет события, а также имитировать нарушения безопасности.



Подсистема контроля целостности

- Развитый механизм контроля целостности системных файлов ОС Linux – IMA (Integrity Measurement Architecture), доступен для ядра версии 2.6.30 и выше. Технология IMA включена не во все дистрибутивы, поэтому для ее использования может потребоваться пересборка ядра.
 - Подсистема IMA отвечает за вычисление хэш-значений файлов и программ при их открытии до того, как их содержимое будет доступно для чтения или выполнения, а также поддерживает проверку их соответствия заранее заданному списку и создание отчетов.
 - При первой установке системы записывается список «хороших» значений хеш-функции. IMA ведет список текущих файловых хэшей, который можно сравнить с первоначальным для обнаружения изменений. Изначально IMA была пассивной защитой.
- В Linux можно проверить целостность установленных пакетов. Это делается командой `rpm --verify --all`, которая сравнивает файлы с оригиналом, проверяя их размер, MD5, разрешения, тип, владельца и группу. Результат — список ошибок с путями файлов.

Функции криптографической защиты

- В ОС Linux поддержка шифрования на уровне файловых систем введена сравнительно недавно – начиная с версии ядра 4.1 и реализуется fscrypt – библиотекой, в которую могут подключаться файловые системы для поддержки прозрачного шифрования каталогов.
- Шифрование в Linux защищает домашнюю папку пользователя. Встроенная система ext4 использует e4crypt, а fscrypt шифрует файлы, но оставляет метаданные открытыми.

Особенности шифрования:

1. Нельзя зашифровать существующие файлы.
 2. Пустой каталог можно пометить как зашифрованный.
 3. После ввода ключа все файлы, каталоги и ссылки в этом каталоге шифруются.
 4. Имена файлов и их содержимое шифруются, но метаданные остаются открытыми.
 5. Открытые файлы видны другим пользователям с правами доступа.
 6. Ключи остаются в памяти до размонтирования системы или удаления кэша.
 7. Нет защиты от расшифровки файлов другого пользователя с неправильным паролем.
- Для доступа к зашифрованным файлам пользователь должен генерировать ключи при каждом запуске системы. Ключи разных пользователей на один пароль совпадают.